

Context Engineering

컨텍스트 엔지니어링

모델에 어떤 정보가 전달되고
어떻게 일관성을 유지하는지를 제어하는
시스템 설계

컨텍스트

엔지니어링

모델에 어떤 정보가 전달되고

어떻게 일관성을 유지하는지를 제어하는

시스템 설계.

서론

프롬프트에서 액션으로의 진화

오케스트레이션 과제

도구 사용의 차세대 영역

AI 엔지니어링의 미래

도구

요약

에이전트란 무엇인가?

01

04

05

07

08

10

11

12

13

25

26

26

27

29

31

35

36

39

40

15

17

18

20

23

컨텍스트 엔지니어링이란 무엇인가?

컨텍스트 윈도우 과제

에이전트를 위한 전략과 작업

에이전트가 컨텍스트 엔지니어링에서 차지하는 위치

에이전트

청킹 전략 가이드

간단한 청킹 전략

고급 청킹 전략

사전 청킹 vs. 사후 청킹

요약

검색

에이전트 메모리 아키텍처

효과적인 메모리 관리를 위한 핵심 원칙

메모리

쿼리 증강

쿼리 재작성

쿼리 확장

쿼리 분해

쿼리 에이전트

목차

프롬프팅 기법

고전적 프롬프팅 기법

고급 프롬프팅 기법

도구 사용을 위한 프롬프팅

프롬프트 프레임워크 사용

컨텍스트 엔지니어링

서론

대규모 언어 모델(Large Language Models, LLMs)로 개발하는 모든 개발자는 결국 같은 벽에 부딪힙니다. 놀라운 능력으로 글을 쓰고, 요약하고, 추론할 수 있는 강력한 모델로 시작합니다. 하지만 실제 문제에 적용하려고 하면 균열이 나타나기 시작합니다. 모델은 개인 문서에 대한 질문에 답할 수 없습니다. 어제 일어난 사건에 대한 지식이 없습니다. 답을 모를 때 자신 있게 내용을 지어냅니다.

메모리에

추가

사용자

장기 메모리

입력

에이전트

단기 메모리

프롬프트

데이터베이스

MCP

답변

1

3

2

4

7

8

6

5

액션 도구

RAG

벡터 검색

(검색)

에이전트 조정

& 의사결정

답변으로

프롬프트 업데이트

모든 컨텍스트를

채팅 기록에 저장

애플리케이션에 기록 감각과

상호작용으로부터 학습할 수 있는

능력을 부여하는 시스템.

메모리

문제는 모델의 지능이 아닙니다. 문제는

모델이 근본적으로 단절되어 있다는 것입니다.

강력하지만 고립된 두뇌로, 특정 데이터,

실시간 인터넷, 심지어 마지막 대화의

기억에도 접근할 수 없습니다. 이러한 고립은

핵심 아키텍처 제한의 직접적인 결과입니다:

바로 컨텍스트 윈도우입니다. 컨텍스트 윈도우는

모델의 활성 작업 메모리로, 현재 작업을 위한

지시사항과 정보를 보관하는 유한한 공간입니다.

모든 단어, 숫자, 구두점이 이 윈도우의 공간을 소비합니다. 화이트보드처럼, 가득 차면 새 지시사항을 위한 공간을 만들기 위해 오래된 정보가 지워지고, 중요한 세부사항이 손실될 수 있습니다.

더 나은 프롬프트를 작성하는 것만으로는 이 근본적인 한계를 해결할 수 없습니다. 모델 주변에 시스템을 구축해야 합니다. 그것이 바로 컨텍스트 엔지니어링입니다. 컨텍스트 엔지니어링은 LLM에 적절한 시간에 적절한 정보를 제공하는 아키텍처를 설계하는 분야입니다. 모델 자체를 변경하는 것이 아니라, 모델을 외부 세계와 연결하는 다리를 구축하고, 외부 데이터를 검색하고, 실시간 도구에 연결하며, 훈련 데이터가 아닌 사실에 기반하여 응답할 수 있는 메모리를 제공하는 것입니다. 이 전자책은 그러한 시스템을 위한 청사진입니다. 뛰어나지만 고립된 모델을 신뢰할 수 있고 프로덕션에 사용 가능한 애플리케이션으로 전환하는 데 필요한 핵심 구성요소를 다룹니다. 이러한 구성요소를 마스터하는 것은 그럴듯한 데모와 진정으로 지능적인 시스템 사이의 차이입니다. 이제 시작해봅시다. 언제 어떻게 정보를 사용할지 조정하는 의사결정 두뇌.

에이전트

지저분하고 모호한 사용자 요청을 정확하고 기계가 읽을 수 있는 의도로 번역하는 기술.

쿼리

증강

LLM을 특정 문서 및 지식 베이스에 연결하는 다리.

검색

모델의 추론을 안내하는 명확하고 효과적인 지시사항을 제공하는 기술.

프롬프팅

기법

애플리케이션이 직접 행동하고 실시간 데이터 소스와 상호작용할 수 있게 하는 도구.

도구

01

02

컨텍스트 엔지니어링

에이전트

대규모 언어 모델로 실제 시스템을 구축하기 시작하면, 정적 파이프라인의 한계에 부딪힙니다. "검색 후 생성"이라는 고정된 레시피는 간단한 검색 증강 생성(Retrieval Augmented Generation, RAG) 설정에는 잘 작동하지만, 작업이 판단, 적응 또는 다단계 추론을 요구하면 무너집니다.

바로 여기서 에이전트가 등장합니다. 컨텍스트 엔지니어링의 맥락에서, 에이전트는 시스템을 통해 정보가 어떻게(그리고 얼마나 잘) 흐르는지 관리합니다. 대본을 맹목적으로 따르는 대신, 에이전트는 자신이 알고 있는 것을 평가하고, 아직 필요한 것을 결정하며, 적절한 도구를 선택하고, 문제가 발생하면 전략을 조정할 수 있습니다.

에이전트는 컨텍스트의 설계자이자 동시에 해당 컨텍스트의 사용자입니다.

그러나 컨텍스트를 잘 관리하는 것은 어렵고, 잘못하면 에이전트가 할 수 있는 다른 모든 것을 빠르게 방해하기 때문에, 에이전트를 안내하는 좋은 관행과 시스템이 필요합니다.

생각 중...

사용자

프롬프트

응답

AI 에이전트

"에이전트"라는 용어는 광범위하게 사용되므로, 대규모 언어 모델(LLMs)로 구축하는 맥락에서 정의해봅시다. AI 에이전트는 다음을 할 수 있는 시스템입니다:

에이전트란 무엇인가?

모든 작업을 스스로 처리하려고 시도하며,

중간 정도로 복잡한 워크플로우에 적합합니다.

단일 에이전트 아키텍처

작업별로 특화된 에이전트에 작업을 분배합니다.

복잡한 워크플로우를 가능하게 하지만

조정 문제를 야기합니다.

다중 에이전트 아키텍처

검색된

정보가

관련성 있음?

각

하위 쿼리

사용자

쿼리

메모리

응답

쿼리를

하위 쿼리로

분해

쿼리 라우팅

및 처리

검색

도구

응답

생성

YES

YES

NO

NO

YES

NO

추가

정보

필요?

이전에

답변함?

정보 흐름에 대한 동적 의사결정을
수행합니다. 미리 정해진 경로를 따르는
대신, 에이전트는 학습한 내용을 기반으로
다음에 무엇을 할지 결정합니다.

여러 상호작용에 걸쳐 상태를 유지합니다.

단순한 Q&A 시스템과 달리, 에이전트는
자신이 한 일을 기억하고 그 기록을
사용하여 미래 결정에 정보를 제공합니다.

도구를 적응적으로 사용합니다.

사용 가능한 도구 중에서 선택하고

명시적으로 프로그래밍되지 않은

방식으로 결합할 수 있습니다.

결과에 따라 접근 방식을

수정합니다. 한 전략이 작동하지

않으면 다른 접근 방식을

시도할 수 있습니다.

1

3

2

4

추론

범례

도구 사용

메모리

03

04

컨텍스트 엔지니어링

이것은 에이전트 시스템을 관리하는 데 있어 가장 중요한 부분 중 하나입니다. 에이전트는 메모리와 도구만 필요한 것이 아니라, 자신의 컨텍스트 품질을 모니터링하고 관리해야 합니다. 이는 과부하를 피하고, 관련 없거나 상충하는 정보를 감지하며, 필요에 따라 가지치기 또는 압축하고, 효과적으로 추론할 수 있을 만큼 컨텍스트 내 메모리를 깨끗하게 유지하는 것을 의미합니다.

컨텍스트 위생

LLM은 컨텍스트 윈도우가 한 번에 보관할 수 있는 정보의 양이 제한되어 있기 때문에 정보 용량이 제한적입니다. 이 근본적인 제약은 에이전트와 에이전트 시스템이 현재 할 수 있는 것을 형성합니다.

에이전트가 정보를 처리할 때마다 다음과 같은 결정을 내려야 합니다:

컨텍스트 윈도우 과제

어떤 정보가 컨텍스트 윈도우에서

활성 상태로 유지되어야 하는지

무엇을 외부에 저장하고

필요할 때 검색해야 하는지

공간을 절약하기 위해

요약하거나 압축할 수 있는 것은 무엇인지

추론 및 계획을 위해

얼마나 많은 공간을 예약해야 하는지

더 큰 컨텍스트 윈도우가 이 문제를 해결한다고 가정하고 싶지만,

이는 사실이 아닙니다. 더 긴 컨텍스트(수십만 또는 심지어 ~100만 토큰)는 실제로 새로운 실패 모드를 야기합니다. 성능은 종종 모델이

최대 토큰 용량에 도달하기 훨씬 전에 저하되기 시작하며,

에이전트는 혼란스러워지고, 환각 비율이 높아지거나, 단순히

정상적으로 할 수 있는 수준에서 수행을 멈춥니다. 이것은

단순한 기술적 제한이 아니라, 모든 AI 앱의 핵심 설계 과제입니다.

컨텍스트 산만

에이전트가 너무 많은 과거 정보(기록,

도구 출력, 요약)에 부담을 느끼고,

새롭게 추론하는 대신 과거 행동을

반복하는 데 과도하게 의존합니다.

컨텍스트 혼란

관련 없는 도구나 문서가 컨텍스트를

가득 채워 모델을 산만하게 하고

잘못된 도구나 지시사항을 사용하게 합니다.

컨텍스트 충돌

컨텍스트 내 모순된 정보가 에이전트를

오도하여, 상충하는 가정 사이에

갈히게 만듭니다.

컨텍스트

컨텍스트

컨텍스트 오염

잘못되거나 환각된 정보가 컨텍스트에

들어갑니다. 에이전트가 해당 컨텍스트를

재사용하고 그 위에 구축하기 때문에,

이러한 오류가 지속되고 복잡됩니다.

컨텍스트 윈도우 크기가 커짐에 따라 발생하거나 증가하기 시작하는

일반적인 오류 유형은 다음과 같습니다:

컨텍스트

05

06

컨텍스트 엔지니어링

품질 검증:

검색된 정보가

일관되고 유용한지

확인합니다.

컨텍스트 요약:

누적된 기록을 주기적으로

요약으로 압축하여

부담을 줄이면서

핵심 지식을 보존합니다.

컨텍스트 가지치기:

전문 가지치기 모델이나

전용 LLM 도구를 사용하여

관련 없거나 오래된

컨텍스트를 적극적으로 제거합니다.

컨텍스트 오프로딩: 모든 것을 활성 컨텍스트에

유지하는 대신, 세부 정보를 외부에 저장하고

필요할 때만 검색합니다.

실패

전략 변경

적응형 검색 전략:

초기 시도가 실패하면 쿼리를 재구성하거나,

지식 베이스를 전환하거나, 청킹 전략을

변경합니다.

동적 도구 선택: 가능한 모든 도구를 프롬프트에

덤프하는 대신, 에이전트는 작업과 관련된 도구만

필터링하고 로드합니다. 이는 혼란을 줄이고

정확도를 향상시킵니다.

다중 소스 통합:

여러 소스의 정보를 결합하고,

충돌을 해결하며,

일관된 답변을 생성합니다.

에이전트를 위한

전략과 작업

에이전트는 동적인 방식으로 추론하고

의사결정을 내리는 능력 덕분에

컨텍스트 시스템을 효과적으로 조정할 수 있습니다.

다음은 에이전트가 컨텍스트를 관리하기 위해

구축되고 사용하는 가장 일반적인 작업입니다.

에이전트는 컨텍스트 엔지니어링 시스템에서 조정자 역할을 합니다. 다른 섹션에서

다루는 기술을 대체하는 것이 아니라, 지능적으로 조정합니다. 에이전트는 초기 검색이

실패했을 때 쿼리 재작성을 적용하거나, 만나는 콘텐츠 유형에 따라 다른 청킹 전략을

선택하거나, 대화 기록을 압축하여 새 정보를 위한 공간을 만들 시점을 결정할 수

있습니다. 정보 관리에 대한 동적이고 컨텍스트에 적합한 의사결정을 내리는 데 필요한

조정 계층을 제공합니다.

컨텍스트 엔지니어링 시스템 내 다양한 유형의 에이전트와 기능:

벡터 DB 에피소드 및

사실 저장소

장기 메모리

단기 메모리
메모리
계획
전문화된
곳으로 라우팅
데이터 수집
선택자
검색기
도구 라우터
쿼리 재작성자
답변
합성기
압축기
작업 메모리
도구 및 API
벡터 DB
지식
컬렉션
웹 및 검색
API
외부 지식
소스
기능 및
지식 소스
관리자
전문화된 에이전트
최종 응답
생성
합성을 위한
컨텍스트 전송
쿼리 정제
도구 결과
반환
검색된
사실
반환
에피소드
메모리
동기화
쿼리 및 업데이트
검색 요청
전송
07
08
컨텍스트 엔지니어링

쿼리 재작성은 원래 사용자 쿼리를 정보 검색에 더 효과적인 버전으로 변환합니다. 단순히 검색 후 읽기를 수행하는 대신, 이제 애플리케이션은 재작성-검색-읽기 접근 방식을 사용합니다. 이 기술은 이상하게 작성된 질문을 시스템이 더 잘 이해할 수 있도록 재구성하고, 관련 없는 컨텍스트를 제거하며, 올바른 컨텍스트와의 매칭을 개선하는 일반적인 키워드를 도입하고, 복잡한 질문을 더 간단한 하위 질문으로 분할할 수 있습니다.

쿼리 재작성

API 호출이 계속

실패할 때 이것을

어떻게 작동시키나요?

query="API 호출 실패,

문제 해결

인증 헤더,

속도 제한, 네트워크

타임아웃, 500 오류"

원시 쿼리

쿼리 재작성자 (LLM)

재작성된 쿼리

RAG 애플리케이션은 쿼리의 구문과 특정 키워드에

민감하므로, 이 기술은 다음과 같이 작동합니다:

불명확한 질문

재구성:

컨텍스트

제거:

키워드

향상:

모호하거나 잘못

형성된 사용자

입력을 정확하고

정보 밀도가 높은

용어로 변환합니다.

검색 프로세스를

혼란스럽게 할 수 있는

관련 없는 정보를

제거합니다.

관련 문서와 매칭될

가능성을 높이는

일반적인 용어를

도입합니다.

쿼리

증강

컨텍스트 엔지니어링의 가장 중요한 단계 중 하나는 사용자의 쿼리를 준비하고

제시하는 방법입니다. 사용자가 정확히 무엇을 묻는지 모르면, LLM은 정확한 응답을

제공할 수 없습니다.

간단하게 들리지만, 실제로는 꽤 복잡합니다. 고려해야 할 두 가지 주요 문제가 있습니다:

쿼리 증강은 파이프라인 시작 부분에서 "쓰레기 입력, 쓰레기 출력" 문제를 해결한다는

것을 기억하세요. 아무리 정교한 검색 알고리즘, 고급 재순위화 모델, 영리한 프롬프트

엔지니어링도 잘못 이해된 사용자 의도를 완전히 보상할 수 없습니다.

사용자는 종종 이상적인 방식으로
챗봇이나 입력과 상호작용하지 않습니다.
파이프라인의 다양한 부분이
쿼리를 다양한 방식으로 처리해야 합니다.
많은 제품 개발자는 요청과 LLM이
질문을 이해하는 데 필요한 모든 추가
정보를 간결하고 완벽하게 구두점이
찍힌 명확한 방식으로 제공하는 쿼리로
챗봇을 개발하고 테스트합니다.
불행히도 실제 세계에서는 사용자와
챗봇의 상호작용이 불명확하고 지저분하며
완전하지 않을 수 있습니다. 견고한
시스템을 구축하려면 이상적인 상호작용만이
아니라 모든 유형의 상호작용을 처리하는
솔루션을 구현하는 것이 중요합니다.
LLM이 잘 이해할 수 있는 질문이
벡터 데이터베이스를 검색하는 데는
최적의 형식이 아닐 수 있습니다.
또는 벡터 데이터베이스에 가장 적합한
쿼리 용어가 LLM이 답변하기에는
불완전할 수 있습니다. 따라서 파이프라인
내의 다양한 도구와 단계에 맞게 쿼리를
증강하는 방법이 필요합니다.

1

2

09

10

컨텍스트 엔지니어링

쿼리 확장은 단일 사용자 입력에서 여러 관련 쿼리를 생성하여 검색을 향상시킵니다. 이 접근 방식은 사용자 쿼리가 모호하거나 잘못 형성되었을 때 또는 키워드 기반 검색 시스템과 같이 더 광범위한 범위가 필요할 때 결과를 개선합니다.

쿼리 확장

그러나 쿼리 확장에는 신중한 관리가 필요한

과제가 있습니다:

쿼리 드리프트:

과도한 확장:

계산

오버헤드:

확장된 쿼리가

사용자의 원래 의도에서

벗어나 관련 없거나

주제를 벗어난 결과를

초래할 수 있습니다.

너무 많은 용어를

추가하면 정밀도가

감소하고 과도한

관련 없는 문서를

검색할 수 있습니다.

여러 쿼리를 처리하면

시스템 지연 시간과

리소스 사용량이

증가합니다.

오픈 소스

NLP 도구

자연어 처리 도구

무료 nlp 라이브러리

오픈 소스 언어 처리 플랫폼

오픈 소스 코드를 가진 NLP 소프트웨어

쿼리 재작성자 (LLM)

원시 쿼리

확장된 쿼리

쿼리 분해는 복잡하고 다면적인 질문을 독립적으로 처리할 수 있는 더 간단하고

집중된 하위 쿼리로 분해합니다. 이 기술은 여러 소스의 정보가 필요하거나 여러

관련 개념을 포함하는 질문에 특히 좋습니다.

쿼리 분해

검색 후, 컨텍스트 엔지니어링 시스템은 모든 하위 쿼리의 결과를 집계하고 합성하여

원래의 복잡한 쿼리에 대한 일관되고 포괄적인 답변을 생성해야 합니다.

컨텍스트 윈도우

분해 단계:

LLM이 원래의 복잡한 쿼리를

분석하고 더 작고 집중된 하위

쿼리로 분해합니다. 각 하위 쿼리는

원래 질문의 특정 측면을 대상으로 합니다.

처리 단계:

각 하위 쿼리는 검색 파이프라인을 통해

독립적으로 처리되어 관련 문서와의 더

정확한 매칭을 가능하게 합니다.

프로세스는 일반적으로 두 가지 주요 단계를 포함합니다:

11

12

컨텍스트 엔지니어링

쿼리 에이전트

쿼리 에이전트는 가장 고급 형태의 쿼리 증강으로, AI 에이전트를 사용하여 전체 쿼리 처리 파이프라인을 지능적으로 처리합니다.

분석: 생성 모델을 사용하여 작업 및 필요한 쿼리를 분석합니다.

동적 쿼리 구성: 사용자 의도와 데이터 스키마를 기반으로 쿼리를 구성합니다.

쿼리 실행: 선택한 컬렉션에 쿼리를 전송합니다.

다중 컬렉션 라우팅: 사용자 질문에 따라 쿼리할 데이터 컬렉션을 결정합니다.

평가: 검색된 정보를 평가하고 필요시 다른 소스를 시도합니다.

응답 생성: 최종 응답을 생성합니다.

검색

대규모 언어 모델은 접근할 수 있는 정보만큼만 우수합니다. 진정으로 지능적인 애플리케이션을 구축하려면 적절한 시간에 적절한 외부 정보를 제공해야 합니다.

칭킹 기법 가이드

칭킹은 검색 시스템 성능을 위한 가장 중요한 결정입니다. 대형 문서를 더 작고 관리 가능한 조각으로 분해하는 프로세스입니다.

컨텍스트 윈도우

- 시스템 프롬프트
- 사용자 쿼리
- 관련 청크들



청킹 전략

두 가지 경쟁하는 우선순위의 균형:

- 검색 정밀도: 작고 집중된 청크
- 컨텍스트 풍부성: 충분히 크고 자기완결적인 청크

간단한 청킹 기법:

- 고정 크기 청킹: 미리 정해진 크기로 분할
- 재귀적 청킹: 우선순위 구분자를 사용하여 분할
- 문서 기반 청킹: 문서 구조를 사용하여 분할

고급 청킹 기법

- Late Chunking: 전체 문서를 먼저 임베딩한 후 청크 분할
- 계층적 청킹: 여러 레벨의 청크 생성
- LLM 기반 청킹: LLM을 사용하여 지능적으로 청크 생성
- 에이전틱 청킹: AI 에이전트가 최적의 전략 선택
- 시맨틱 청킹: 의미 기반으로 텍스트 분할



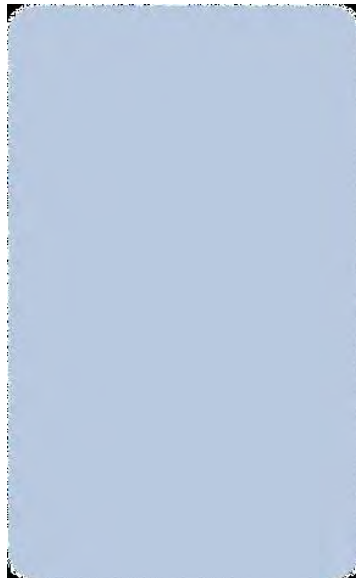
사전 청킹 vs. 사후 청킹

사전 청킹:

- 모든 데이터 처리를 미리 수행
- 검색이 매우 빠름
- 청킹 전략이 고정됨

사후 청킹:

- 사용자 쿼리에 따라 실시간 청킹
- 높은 유연성
- 지연 시간 추가



청킹 전략 선택 가이드

전략별 복잡도와 적합한 사용 사례:

- 고정 크기: 간단한 문서, 속도 중요
- 재귀적: 구조 유지하면서 효율성 필요
- 문서 기반: 구조화된 파일
- 시맨틱: 기술 문서, 학술 문서
- LLM 기반: 복잡한 텍스트
- 에이전틱: 맞춤 전략 필요
- Late Chunking: 전체 컨텍스트 인식 필요
- 계층적: 요약과 세부사항 모두 필요

요약: 검색 시스템의 효과는 청킹 전략과 아키텍처 패턴의 선택에 달려 있습니다.

프롬프팅 기법

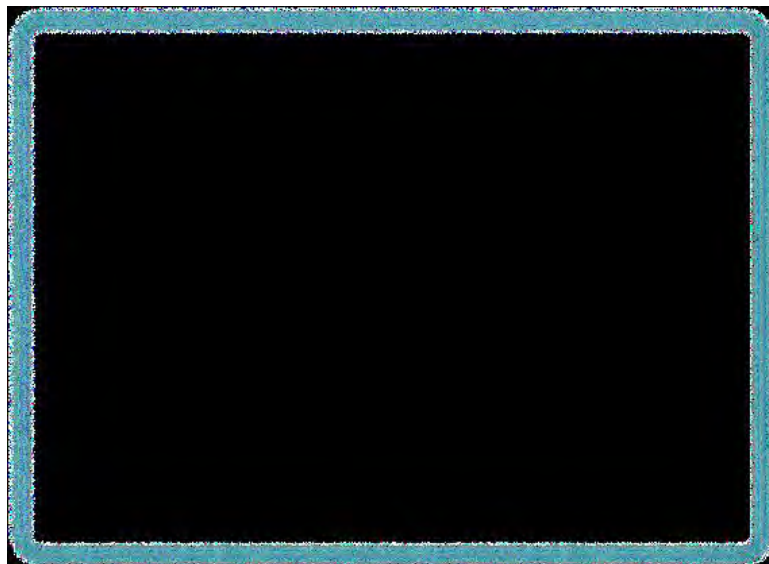
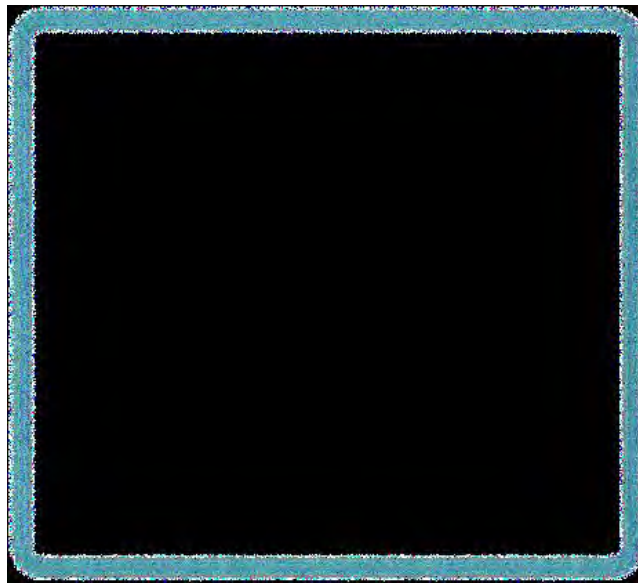
프롬프트 엔지니어링은 LLM에 제공하는 입력(프롬프트)을 설계, 개선, 최적화하는 관행입니다. 프롬프트의 품질과 효과성은 LLM의 응답에 크게 영향을 미칩니다.

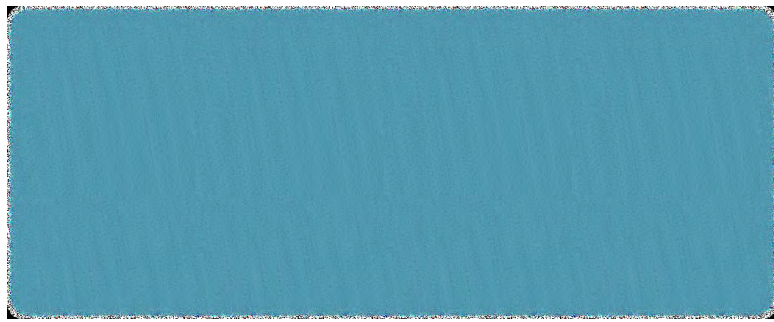
고전적 프롬프팅 기법:

- Chain of Thought: 단계별 추론
- Few-Shot Prompting: 예시를 통한 학습

팁 #1: CoT 추론을 사용 사례에 맞게 구체화

팁 #2: 효율성을 위해 간결한 추론 형식 사용





고급 프롬프팅 전략

- Tree of Thoughts (ToT): 여러 추론 경로를 병렬로 탐색
- ReAct Prompting: 추론과 행동을 결합

도구 사용을 위한 프롬프팅:

- 명확한 도구 설명 제공
- 사용 조건 정의
- Few-shot 예시 포함

효과적인 도구 설명 작성:

- 능동 동사 사용
- 입력 사항 명시
- 출력 설명
- 제한사항 언급

프롬프트 프레임워크: DSPy, Llama Prompt Ops 등을 고려할 수 있지만 필수는 아님



메모리

메모리는 에이전트에 생명을 불어넣는 요소입니다. 메모리 없이는 LLM은 상태가 없는 텍스트 프로세서에 불과합니다.

에이전트 메모리 아키텍처:

- 단기 메모리: 컨텍스트 윈도우 내의 즉각적인 작업 공간
- 장기 메모리: 외부에 저장되어 필요시 검색

컨텍스트 오프로딩: 제한된 토큰 공간을 확보하기 위해 정보를 외부 도구나 벡터 데이터베이스에 저장하는 관행.

Andrej Karpathy의 비유:

- 컨텍스트 윈도우 = RAM
- LLM = CPU

하이브리드 메모리 설정

대부분의 현대 시스템은 단기 메모리와 장기 메모리를 혼합하여 사용합니다.

추가 레이어:

- 작업 메모리: 특정 다단계 작업을 위한 임시 보관 영역
- 절차적 메모리: 루틴을 학습하고 마스터

메모리 유형:

- 에피소드 메모리: 과거 사건 및 상호작용
- 시맨틱 메모리: 일반 지식 및 사실
- 절차적 메모리: 학습된 루틴

프로세스:

1. 검색: 관련 지식 검색
2. 작업 상태 저장: 임시 스크래치패드에 저장
3. 작업 컨텍스트 회상: 다단계 프로세스 계속
4. 메모리 저장: 중요한 정보 영구 저장

효과적인 메모리 관리를 위한 핵심 원칙

1. 메모리 정리 및 개선

- 주기적으로 장기 저장소 스캔
- 중복 항목 제거
- 관련 정보 병합
- 오래된 사실 삭제
- 최신성과 검색 빈도를 기준으로 정리

2. 저장 내용 선택

- 모든 상호작용을 영구 저장할 필요 없음
- 품질과 관련성을 기준으로 필터링
- LLM이 "반성"하여 중요도 점수 부여
- 컨텍스트 오염 방지

3. 작업에 맞는 아키텍처 조정

- 만능 솔루션은 없음
- 고객 서비스 봇: 강력한 에피소드 메모리 필요
- 금융 분석 에이전트: 도메인 특화 시맨틱 메모리 필요
- 가장 간단한 접근 방식부터 시작하여 점진적으로 개선

검색 기술 마스터

효과적인 메모리는 저장량보다 적절한 정보를 적시에 검색하는 능력에 달려 있습니다.

고급 기법:

- 재순위화(Reranking): LLM을 사용하여 검색 결과의 관련성에 따라 재정렬
- 반복 검색: 여러 단계에 걸쳐 검색 쿼리를 개선/확장

쿼리 에이전트와 개인화 에이전트는 이러한 기능을 기본적으로 제공:

- 여러 컬렉션에 걸친 검색
- 사용자 선호도 및 상호작용 기록을 기반으로 재순위화

메모리는 단순한 수동 저장이 아니라 능동적이고 관리되는 프로세스입니다.

목표는 무엇을 기억하고, 무엇을 잊고, 과거를 사용하여 미래에 대해 추론하는 방법을 아는 에이전트를 구축하는 것입니다.

도구

메모리가 에이전트에 자아 감각을 부여한다면, 도구는 초능력을 부여합니다.

LLM은 훌륭한 대화 상대이자 텍스트 조작자이지만 버블 안에 살고 있습니다.

도구는 LLM 에이전트를 외부 세계와 연결하여 실제 세계에서 직접 행동하고 작업을 수행하는 데 필요한 정보를 검색할 수 있게 합니다.

프롬프트에서 액션으로의 진화:

- 초기: 프롬프트 엔지니어링으로 명령처럼 보이는 텍스트 생성
- 현재: 함수 호출(도구 호출) - 구조화된 JSON 출력

가능성:

- 간단한 도구: 여행 에이전트가 search_flights 도구 사용
- 도구 체인: 복잡한 요청에 대해 여러 도구를 연결
(예: find_flights, search_hotels, get_local_events)

반복 사이클: Thought-Action-Observation 루프

```
[15:43:27] - GLOWE INFO - User prajwal@weaviate.io authenticated
INFO: ('127.0.0.1', 6060) - "websocket /ws/chat" (accepted)
INFO: - GLOWE INFO - Sending response to frontend: status
[15:43:30] - GLOWE INFO - Step 1: Initializing tree for user ad85db1b-719a-5635-8a87-cf3e6339484c with chat_id db6b7d7b-ea8d-4487-a9f9-9ffe8bab8d4
[15:43:30] - GLOWE INFO - Step 2: Configuring tree settings
INFO: - GLOWE INFO - Initialised tree with the following decision nodes:
- user {}
[15:43:38] - GLOWE INFO - Step 3: Fetching current product and stack
[15:43:38] - GLOWE INFO - Step 4: Setting up metadata
[15:43:38] - GLOWE INFO - Adding new GLOWE metadata
[15:43:38] - GLOWE INFO - Step 5: Configuring tools
[15:43:38] - GLOWE INFO - Adding product agent tool
[15:43:38] - GLOWE INFO - Adding similar products agent tool
[15:43:38] - GLOWE INFO - Adding stack agent tool
[15:43:38] - GLOWE INFO - Configured tools: 'forced_text_response', 'text_response', 'cited_summarize', 'product_agent', 'similar_products_agent', 'stack_agent'
[15:43:38] - GLOWE INFO - Step 6: Starting conversation loop
INFO: Style: Skincare professional and expert recommender who is friendly, empathetic, and patient
INFO: Agent description:
Identity & Context
You are GlowE, a warm, professional virtual skincare expert focused on Korean-beauty (K-beauty) routines.
The GlowE app is powered by Elysia (agentic decision-tree with chain-of-thought self-healing) and a Weaviate
Query Agent for context-aware product and ingredient searches.
2. Mission
Empower every user to create safe, enjoyable, and effective K-beauty "stacks" (ordered routines) matched to
their skin type, concerns, sensitivities, and goals.
```

```
Current Decision
Node: root
Decision: product_agent
Reasoning: The user is asking for a product recommendation (specifically, a cleanser).
According to the instruction, I should use the "product_agent" tool for product recommendations.
I should also use the user's skin profile when making recommendations.
The user's skin type is 22.0 (dry), they have redness and wrinkles as concerns, and they want to hydrate and glow up.
I previously attempted to use the "product_agent" tool, but it resulted in an error.
I will try again, but this time I will simplify the search query to avoid the error.
I will search for a cleanser that is suitable for dry skin and helps with hydration.

Original Query
cleanser for dry skin and hydration

Final Answer
For dry skin with a need for hydration, I recommend these gentle, moisturizing Korean cleansers that contain multiple types of hyaluronic acid:
1. MISSHA Super Aqua Ultra Hyalon Foaming Cleanser - Contains over 10 kinds of hyaluronic acid to deeply hydrate and maintain skin moisture.
2. COSRX Hydr8N Triple Hyaluronic Moisturizing Cleanser - Foam cleanser with three hyaluronic acid types and Vitamin B5 to replenish moisture.
3. Torriden DIVE-IN Low Molecular Hyaluronic Acid Cleansing Foam - A pH-balanced foam cleanser with eight kinds of micro-molecular hyaluronic acid promoting a supple, hydrated feel.
All three provide effective cleansing combined with hydrating benefits that won't strip your skin. Let me know if you'd like a cleanser recommendation!

Searches Executed 1/1
QueryResultWithCollection(
  queries=[{"cleanser": "gentle hydrating moisturizing dry skin"}],
  filters=[{"textProperty": "filter(property_name='category', operator=ComparisonOperator.EQUALS: '=', value='Face Washes')"}],
  filter_operators='AND',
  collection='Products_v2'
)
```

```

    User prompt
    what's the best cleanser for my skintype?

[15:43:48] - GLOW INFO - Unknown result type: tree_update
[15:43:48] - GLOW INFO - Sending response to frontend: status
Assistant response
I'm using your skin profile to find the best cleanser for your dry skin, redness, and wrinkles, while also helping with hydration and glow.

[15:43:48] - GLOW INFO - Sending response to frontend: text
[00/023/025: 15:43:18] INFO Time taken for Decision Node (decision_node): 4.59 seconds
:327
Current Decision
Node: root
Decision: product_agent
Reasoning: The user is asking for a product recommendation (specifically, a cleanser).
According to the instruction, I should use the "product_agent" tool for product recommendations.
I should also use the user's skin profile when making recommendations.
The user's skin type is 22.0 (dry), they have redness and wrinkles as concerns, and they want to hydrate and glow up.
Therefore, I should search for a cleanser that is suitable for dry skin, addresses redness and wrinkles, and helps with hydration and glow.
OBJECT1.PY
```


오케스트레이션 과제

에이전트에게 도구를 제공하는 것은 쉽지만, 신뢰성 있고 안전하며 효과적으로 사용하게 하는 것이 진정한 과제입니다.

핵심 오케스트레이션 단계:

1. 도구 발견

- 사용 가능한 도구 목록과 설명을 시스템 프롬프트에 제공
- 도구 설명의 품질이 매우 중요

2. 도구 선택 및 계획 (Thought)

- 도구가 필요한지 결정
- 복잡한 작업의 경우 여러 도구를 체인으로 연결

3. 인수 공식화 (Action)

- 도구에 전달할 인수 결정
- 구조화된 요청 또는 API 호출 생성
- 자가 치유(Self-healing): 오류 발생 시 자체 수정

4. 반영 (Observation)

- 도구 실행 후 출력을 컨텍스트 윈도우로 피드백
- 다음 단계 결정: 성공? 다른 도구 필요? 사용자에게 명확화 요청?

Thought-Action-Observation 사이클은 현대 에이전틱 프레임워크의 기본 추론 루프를 형성합니다.

```
Current Decision
Node: root
Decision: text_response
Reasoning: The user is asking for the best cleanser for their skin type.
I have already used the 'product_agent' twice to find cleansers for the user's skin type (dry, redness, wrinkles, hydration, glow).
The second attempt was successful, and I have a list of 3 cleansers to recommend to the user.
I should now respond to the user with the list of cleansers.
I should use the 'text_response' tool to respond to the user.
I should also include the user's skin profile in the response.

Assistant response
I recommend these gentle, moisturizing Korean cleansers that contain multiple types of hyaluronic acid or soothing ingredients:
1. MISSHA Super Aqua Ultra Hyalon Foaming Cleanser - Contains over 10 kinds of hyaluronic acid to deeply hydrate and maintain skin moisture while thorough skin.
2. COSRX Hydrium Triple Hyaluronic Moisturizing Cleanser - Foam cleanser with three hyaluronic acid types and Vitamin B5 to replenish moisture and form
3. Torriden DEVE-IN Low Molecular Hyaluronic Acid Cleansing Foam - A pH-balanced foam cleanser with eight kinds of micro-molecular hyaluronic acid, removing promoting a supple, hydrated feel.
All three provide effective cleansing combined with hydrating benefits that won't strip your skin. Let me know if you'd like a cleanser recommendation concerns!

[15:44:32] - GLINE INFO - Sending response to frontend: text
[15:44:34] - GLINE INFO - Title: Best Korean Cleansers for Dry, Red, Wrinkled Skin with Hyaluronic Acid created.
[15:44:34] - GLINE INFO - Sending response to frontend: completed
[15:44:34] - GLINE INFO - Model identified overall goal as completed!
```

도구 사용의 차세대 영역

Model Context Protocol (MCP):

- Anthropic이 2024년 말에 도입한 표준 프로토콜
- "AI를 위한 USB-C"
- AI 애플리케이션을 외부 데이터 소스 및 도구에 연결하는 범용 표준

기존 통합 vs MCP:

- 기존: NxM 연결 (각 모델이 각 데이터 소스와 맞춤 통합 필요)
- MCP: N+M 연결 (모델과 데이터 소스가 MCP와 한 번만 통합)

장점:

- 맞춤 통합을 위한 $M \times N$ 문제를 $M + N$ 문제로 단순화
- JSON-RPC 기반 클라이언트-서버 통신
- 구성 가능하고 표준화된 아키텍처

이는 AI 도구의 미래를 나타내며, 엔지니어의 역할을 맞춤 통합 작성에서 적응형 시스템 오케스트레이션으로 변화시킵니다.

요약

컨텍스트 엔지니어링은 단순히 LLM을 프롬프팅하거나 검색 시스템을 구축하는 것 이상입니다. 다양한 사용 사례와 사용자에게 걸쳐 안정적으로 작동하는 상호 연결된 동적 시스템을 구축하는 것입니다.

핵심 구성요소:

- 에이전트: 시스템의 의사결정 두뇌
- 쿼리 증강: 지저분한 요청을 실행 가능한 의도로 번역
- 검색: 모델을 사실 및 지식 베이스에 연결
- 메모리: 기록 감각과 학습 능력 제공
- 도구: 실시간 데이터 및 API와 상호작용할 수 있는 실행력 제공

우리는 모델과 대화하는 프롬프터에서 모델이 사는 세계를 구축하는 아키텍트로 진화하고 있습니다. 최고의 AI 시스템은 더 큰 모델이 아니라 더 나은 엔지니어링에서 탄생합니다.

Weaviate와 함께 차세대 AI 애플리케이션을 구축할 준비가 되셨나요?

